

VCF-AFA Project Handover Summary for Rohit

1. Purpose of This Handover

This document gives Rohit the clean current-state view of the VCF-AFA project.

It intentionally avoids obsolete implementation history, failed design experiments, and older architecture assumptions. The goal is to give Rohit enough context to continue development without accidentally reviving old decisions or expanding the scope in the wrong direction.

The current accepted implementation baseline is:

VCF-AFA v1.39

Future development should start from **v1.39** unless Keith explicitly instructs otherwise.

2. Project Objective

VCF-AFA is an advisory application for assessing VMware Cloud Foundation environments against alternative infrastructure platform options.

The application supports customer advisory workshops by helping consultants:

- Capture customer business, strategic, sovereignty, and technical requirements.
- Compare a selected VCF baseline against alternative comparator platforms.
- Evaluate functional parity at a granular level.
- Identify full parity, partial parity, no parity, workarounds, risks, and impacts.
- Generate executive-level feasibility and shortlisting outputs.
- Support repeatable, evidence-backed advisory conclusions.

The application is not intended to be a lifecycle management system, migration automation product, generic product comparison tool, or vendor marketing asset.

Its purpose is advisory shortlisting and migration-target feasibility assessment.

3. Current Accepted Baseline

The accepted current baseline is:

VCF-AFA v1.39

v1.39 should be treated as the only active implementation baseline.

v1.39 includes

- Active Data Mode.
- Demo Mode.
- Executive Decision View.
- Comparator feasibility visualization.
- Comparator selection behavior.
- Comparator color synchronization.
- VCF Baseline Version chip behavior.
- Saved AssessmentProfile handling.
- Resume Assessment behavior.
- Breadcrumb navigation.
- KB upload recognition.
- KB diagnostics.
- Ingestion Regression Guard.
- Technical Requirements Capture workflow.
- Strategic Considerations workflow.
- Results and export behavior.
- Stronger executive-grade card styling across workflow screens.
- Stable rendering approach without broad background self-healing timers or broad persistent observer patchers.

Baseline rule

Do not restart from older builds.

Do not reintroduce earlier experimental architectures.

Do not use older failed UI or rendering approaches as the foundation for new work.

4. Repository / Source Status

The old repository structure should not be treated as active.

Rohit should confirm the current active source location, packaging method, and working file structure before making changes.

Do not assume the previous repository folder model is still the operating structure.

The important principle is not the old folder layout. The important principle is that future app inputs, KBs, schemas, configuration files, and exports must have clear data contracts and be easy to validate, ingest, and promote.

5. Core Architecture Direction

The project should be understood as a governed advisory platform made up of several logical layers.

5.1 Assessment Application

The main VCF-AFA assessment app should remain focused on:

- Customer workshop capture.
- VCF baseline selection.
- Comparator selection.
- Requirement capture.
- Strategic consideration capture.
- Feasibility scoring.
- Executive decision output.
- Export/report generation.
- Repeatable evaluation using definitive rules and weights.

It should not become the upstream research engine for discovering new product versions.

5.2 Knowledge Base Inputs

The assessment app should ingest approved KB files and configuration inputs.

The app may require several files to be uploaded during initialization. This is acceptable if it keeps the app cleaner, more governed, and less hardcoded.

Candidate initialization files include:

```
All VCF version KB files
All latest comparator KB files
Latest Canonical Model file
Strategic Considerations list
Rules / weighting / configuration files
Any other required app parameter files
```

The app should make upload requirements explicit and validate that all required files are present before assessment work begins.

5.3 Separate Discovery / Seeding Application

Discovery of new VCF versions and comparator versions should be developed as a separate application.

That separate app should:

- Discover new VCF and comparator versions.

- Generate updated seeded KBs.
- Prepare KBs for human review.
- Support promotion into assessment-ready KBs.
- Output files in the exact structure required by the assessment app.

The assessment app should consume approved KBs. It should not be overloaded with automated discovery, evidence harvesting, and seed generation.

This separation is important. Combining assessment, research discovery, KB generation, and governance into one app would make the system harder to validate and harder to defend.

6. Comparator Strategy

The comparator model is platform-centric, not vendor-centric.

The application compares practical platform capability and functionality, not vendor brand positioning.

6.1 Core Comparator Set

Current development should focus on the Core comparator set:

```
VCF
NCP_AHV
AzureLocal
OpenShiftVirtualization
OpenStackKVM
ProxmoxVE
```

These are the primary comparators for active development, evidence mapping, scoring, and advisory output.

6.2 Extended Comparator Set

The extended comparator set exists for possible future expansion:

```
XCPng
Harvester
HyperV
CitrixHypervisor
ScaleComputingHC3
```

These should not distract from the Core comparator work.

Ruthless guidance: do not expand active comparator scope until the Core comparator evidence model is stable. More comparators without stronger evidence will create volume, not value.

7. Comparator Versioning Principle

Not every comparator should be represented by a single product version.

Some comparator solutions are compositions of multiple products, each with its own development lifecycle, release cadence, and compatibility matrix.

For composite comparators, the app should support an evaluation-period model.

Example:

Comparator: OpenStackKVM Composite
Evaluation Basis: Q2-2026

This means the comparator reflects the compiled functionality set that was evaluated during that period, not a single clean product release.

Practical implication

The app should support both:

Single-product version basis

and:

Composite evaluation-date / evaluation-period basis

This is especially important for platforms assembled from multiple components, distributions, add-ons, or ecosystem tools.

Do not force every comparator into a single version-number field. That will create misleading precision.

8. Canonical Model Concept

The canonical model is the structured assessment model used to keep customer workshops manageable while preserving enough technical granularity for real comparator evaluation.

The current canonical model candidate is:

CN v0.3

CN v0.3 should be treated as the active draft candidate, not as fully final until validated.

8.1 Target Scale

The current planning target is approximately:

12 VCF products / components
80 major features
240 major functionalities

This structure is deliberate.

Too little detail would make comparator scoring weak.

Too much detail would make customer workshops unusable.

8.2 Customer-Facing vs Scoring Structure

The app should distinguish between customer-facing structure and scoring structure.

Customer-facing assessment should primarily use:

Product / Component
Feature

Detailed analysis and scoring should use:

Functionality_ID

Functionality_ID is the key unit for evidence, parity mapping, scoring, aggregation, and comparator analysis.

8.3 Current 12 VCF Product / Component Areas

The current canonical model concept uses these 12 top-level VCF areas:

1. VCF Platform / Fleet / Workload Domains
2. VCF Installer / SDDC Manager / Lifecycle Services
3. vSphere Compute & Management
4. vSAN & Storage Services
5. NSX Networking / Edge / VPC Services
6. NSX Security & Firewall Services
7. VCF Operations & Observability
8. VCF Automation & Self-Service
9. vSphere Kubernetes Service & Application Platform
10. HCX / Import / Converge / Workload Mobility
11. Protection / Recovery / Resilience
12. VCF Private AI Services

8.4 Add-ons and Integrations

Official add-ons, advanced services, internal components, and external integrations should not inflate the top-level product/component list unless there is a strong assessment reason.

They should be handled through a supporting add-on or integration structure.

Ruthless guidance: the 12-area structure is a control mechanism. Do not keep adding top-level categories because a product or feature feels important. The model must stay workshop-usable.

9. Assessment Hierarchy

The assessment hierarchy is:

Platform
Domain
Capability
Feature
Functionality

Scoring should happen at Functionality level first.

Results should then aggregate upward:

Functionality → Feature → Capability → Domain → Platform

This matters because executive outputs must be traceable back to evidence-backed functional detail.

10. Parity Model

Each functionality comparison should resolve to one of:

Full Parity
Partial Parity
No Parity

10.1 Full Parity

The comparator supports the required functionality sufficiently for the assessed use case.

10.2 No Parity

The comparator does not provide a meaningful equivalent for the required functionality.

10.3 Partial Parity

Partial parity must be treated seriously.

It is not a vague middle score.

Partial parity should include:

Workaround availability
Workaround description
Workaround complexity: Low / Medium / High
Technical impact
Operational impact
Commercial impact
Sovereignty impact, where relevant
Advisory notes
Evidence links

Ruthless guidance: partial parity without impact context is not useful. It only hides risk.

11. Requirement Capture Model

Customer requirements should be captured at multiple levels:


```
Capability
Feature
Functionality
```

Higher-level ratings should cascade downward by default.

Lower-level overrides must be supported.

The effective requirement used by the feasibility engine should be calculated after cascade and override resolution.

Requirement importance levels

The model should support a pattern such as:

```
Required
Important
Nice-to-have
Not required
```

The exact enum values should be governed through configuration.

12. AssessmentProfile Concept

AssessmentProfile is a first-class app object.

It should not be treated as an external spreadsheet workaround.

Assessment profiles should support:

```
Save
Export
Import
Resume
Multi-session workshop continuity
Customer context
Selected VCF baseline
Selected comparators
Requirement ratings
Requirement overrides
Business priorities
Strategic considerations
```

```
Sovereignty requirements
Deployment requirements
Weights
Notes
Assessment status
Session metadata
```

This is critical because real advisory workshops may span multiple sessions.

A consultant must be able to resume without losing context or re-entering data.

13. Administration / Configuration Page

The app should be enhanced with an Administration or Configuration page.

This page should expose the logic that controls evaluation behavior.

It should include, at minimum:

```
Feasibility rules
Assessment weightings
Scoring logic
Rulesets
Thresholds
Requirement importance mappings
Comparator assumptions
Aggregation rules
Risk burden logic
Strategic consideration weighting
```

The business requirement is repeatability.

Given the same inputs, KBs, rules, and weights, the app must produce the same result each time.

Ruthless guidance: if rules and weights remain hidden in code, the tool will be hard to audit, hard to govern, and hard to defend in front of customers.

14. Current Application Functionality

The current application supports the following functional areas.

14.1 Welcome / Initialization

Starts the workflow and prepares the assessment context.

This area should evolve to support multi-file initialization where required.

14.2 KB Upload and Diagnostics

Supports KB upload recognition and validation.

Includes diagnostic behavior to confirm that required knowledge inputs have been recognized and ingested correctly.

14.3 Ingestion Regression Guard

Protects against silent data-load failure.

This is important because bad or incomplete KB ingestion would invalidate assessment results.

14.4 Assessment Basics

Captures core customer and assessment context.

Includes VCF baseline context.

14.5 Strategic Considerations

Captures business, strategic, sovereignty, and deployment considerations.

These should influence feasibility interpretation, not just sit as passive notes.

14.6 Technical Requirements Capture

Captures customer technical requirements against the canonical model.

Supports structured requirement gathering across the assessment hierarchy.

14.7 Weighting

Supports weighting of assessment dimensions so outputs reflect the customer's priorities.

Weighting should eventually be governed through the Administration / Configuration page.

14.8 Post-Assessment Review

Allows review of assessment inputs and context before final output.

14.9 Results / Executive Decision View

Provides the primary advisory output.

The Executive Decision View includes:

- Comparator feasibility visualization
- Comparator selection behavior
- Decision map
- Risk and feasibility interpretation
- Watch-area insights
- Technical fit interpretation
- Strategic alignment interpretation
- VCF baseline context
- Executive-grade card layout

14.10 Active Data Mode

Active Data Mode is the production path.

It must remain the source of truth for real assessments.

14.11 Demo Mode

Demo Mode is a permanent feature.

It is used for demonstration, storytelling, and UX validation.

Demo Mode must not corrupt or override Active Data Mode behavior.

14.12 Export / Reporting

The app supports results and assessment exports.

Exports should remain traceable to inputs, rules, weights, KB versions, comparator evaluation basis, and evidence where available.

15. UI / UX Direction

The accepted design language is:

- Executive-grade
- Structured

- Calm
- Clearly contained
- Visible card boundaries
- Light contextual fills
- Strong card headers
- Consistent workflow screen styling
- Clear distinction between technical and strategic interpretation

Avoid subtle changes that are technically present but not visibly useful.

If the user cannot see and understand the improvement immediately, it is not a meaningful improvement.

16. Rendering and Stability Principles

The app should avoid broad background patching and hidden self-healing UI behavior.

Preferred approach:

- Explicit render hooks
- Controlled navigation events
- Controlled upload/change/click events
- Render-signature guarding where useful
- Preserved scroll position across renders
- Scoped diagnostics
- No broad persistent MutationObserver patchers
- No recurring production self-healing loops

The app is used in advisory workshops. Screen jumps, unexpected re-renders, or unstable visual behavior will undermine trust quickly.

17. Sovereignty and Deployment Context

The broader portfolio context includes sovereign infrastructure advisory.

The app should be capable of capturing and evaluating requirements related to:

- Data placement
- Data routing
- Administrative access
- Security policy control
- Configuration control
- Keystore control

- IAM component control
- Workload control
- Deployment model
- Operational control

Target environments may include:

- Far Edge
- Near Edge
- Central Compute
- On-premises environments
- Self-managed private cloud environments

Sovereignty should influence feasibility and advisory interpretation, especially where comparator architectures differ materially.

18. Active Backlog and Forward Ideas

18.1 Validate Canonical Model v0.3

Validate the 12 / 80 / 240 structure against real advisory needs.

Check for:

- Customer-facing clarity
- Technical completeness
- Evidence mapping readiness
- Comparator scoring usefulness
- Workshop usability

18.2 Build Core Comparator Evidence

Prioritize evidence-backed mapping for:

- VCF
- NCP_AHV
- AzureLocal
- OpenShiftVirtualization
- OpenStackKVM
- ProxmoxVE

Do not prematurely expand into Extended comparators.

18.3 Support Composite Comparator Evaluation Periods

Enhance the comparator metadata model to support:

```
Product version
Composite solution evaluation date
Composite solution evaluation period
```

Example:

```
Evaluation Basis: Q2-2026
```

This prevents misleading single-version representation for composite platforms.

18.4 Add Administration / Configuration Page

Expose rules, weights, thresholds, and scoring logic.

This is a governance requirement, not just a UX enhancement.

18.5 Build Separate Discovery / Seeding App

Develop a separate app for:

```
New VCF version discovery
New comparator version discovery
Seeded KB generation
Human review
Promotion to assessment-ready KB
Output formatting aligned to assessment app ingestion
```

18.6 Strengthen Multi-File Initialization

Allow the assessment app to initialize from several uploaded files if needed.

The app should validate file completeness and compatibility before assessment begins.

18.7 Improve Watch Area Insight Presentation

The Watch Area Insight module should clearly distinguish:

Technical Fit
Strategic Alignment

These labels should use subtle executive-grade pill styling and behave consistently in both Active Data Mode and Demo Mode.

18.8 Refine Executive Decision Map Helper Copy

Target helper wording:

Comparator feasibility results - Select individual comparator for details

And:

Bubble Placement = Overall feasibility | Bubble Size = Risk Burden

18.9 Continue Report / Export Hardening

Exports should include enough context to explain how conclusions were reached.

They should preserve:

Customer context
Selected VCF baseline
Selected comparators
Comparator evaluation basis
Ruleset used
Weights used
KB inputs used
Evidence references where available
Final feasibility output
Watch areas
Advisory notes

18.10 Strengthen Schema and Data Quality Controls

Continue improving:

Required field validation
Enum consistency
Parity classification validation
Evidence link validation

Import/export compatibility
KB ingestion checks
AssessmentProfile compatibility
Configuration compatibility

19. Development Guardrails

Rohit should follow these unless Keith explicitly changes direction.

Start from v1.39

v1.39 is the accepted implementation baseline.

Increment versions

Each new app build or artifact package should increment the relevant version number.

Protect Active Data Mode

Active Data Mode is the production path.

No feature should work only in Demo Mode.

Preserve Demo Mode

Demo Mode remains useful and permanent, but it must not become the production data path.

Keep comparator scope controlled

Focus on Core comparators first.

Extended comparators come later.

Keep scoring evidence-backed

Do not invent parity conclusions to complete a matrix.

Unknown is better than false precision.

Keep the canonical model usable

Do not over-granularize the model.

The tool must support real workshops, not become an academic taxonomy.

Make rules visible and repeatable

Scoring, weighting, and feasibility logic should be governed and eventually configurable.

Separate assessment from discovery

The assessment app should consume approved KBs.

Discovery and seeded KB generation should be handled separately.

Avoid hidden rendering patchwork

Do not use broad background observers or recurring self-healing loops as a substitute for clean render logic.

20. Immediate Recommended Next Steps for Rohit

1. Take v1.39 as the only valid implementation baseline.
 2. Confirm the current active source location and file structure.
 3. Review the current app behavior in both Active Data Mode and Demo Mode.
 4. Validate the CN v0.3 canonical model concept.
 5. Confirm the required input files for app initialization.
 6. Define the target metadata model for comparator versioning and evaluation periods.
 7. Draft the Administration / Configuration page model.
 8. Prioritize evidence mapping for the Core comparator set.
 9. Define the data contract between the future Discovery / Seeding app and the Assessment app.
 10. Strengthen validation before adding more comparators or UI features.
-

21. What Rohit Should Not Do

Do not restart the architecture.

Do not build from older versions.

Do not assume the old repository structure is still active.

Do not expand comparator scope before the Core set is stable.

Do not treat Demo Mode as the production data path.

Do not hide scoring logic permanently in code.

Do not force composite comparators into misleading single version numbers.

Do not create parity conclusions without evidence.

Do not overbuild discovery functionality inside the assessment app.

Do not add so much detail to the canonical model that customer workshops become impractical.

Do not make subtle UI changes and call them progress.

22. Bottom Line

VCF-AFA is currently positioned as a governed advisory application for VCF comparator assessment.

The active baseline is:

v1.39

The strategic direction is:

Assessment app consumes governed inputs.
Discovery and seeded KB generation become a separate app.
Comparator analysis remains platform-centric.
Core comparators come first.
Canonical model remains controlled and workshop-usable.
Scoring starts at Functionality level and aggregates upward.
Partial parity requires workaround and impact context.
Rules, weights, and thresholds should become visible and configurable.
Composite comparators may need evaluation periods instead of single version numbers.
Active Data Mode remains the production path.
Demo Mode remains a demonstration and design-support path.

Rohit's first priority should be to stabilize the model and data contracts, not to expand the UI or comparator list.

The strongest next move is to validate the canonical model, define the initialization input set, and harden the evidence-backed Core comparator KBs. That is what will turn the app from a good prototype into a defensible advisory platform.